PROBLEM 7 · SOLUTION

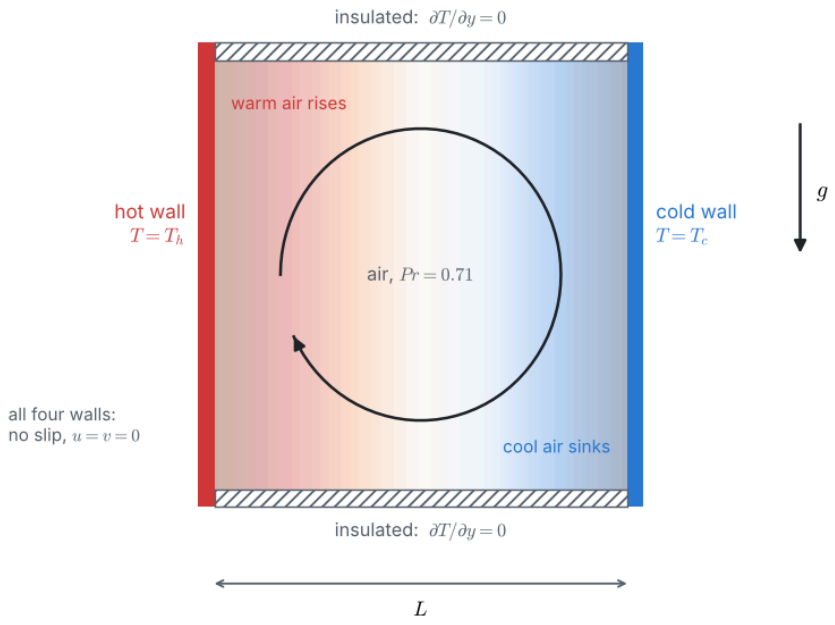
Natural convection in a heated square cavity

Step-by-step solution with the Finite Difference Method

Plan. We follow the 7 pillars of CFD: geometry → grid → physics → numerical scheme → solution setup → compute → post-processing. All code below is plain Python (lists and loops only; `math` and `time` from Python itself). It is the code of the notebook `03_fdm_solver.ipynb`, where it can be run cell by cell.

Step 1 · Geometry and data

A square cavity full of air: hot left wall, cold right wall, insulated top and bottom. Warm air rises along the hot wall, cool air sinks along the cold wall, and a slow clockwise circulation carries heat across the cavity.



The cavity and its boundary conditions.

In dimensionless form (lengths in L, velocities in α/L, $T = (T_{dim} - T_c)/(T_h - T_c)$) only two numbers remain: the **Rayleigh number** $Ra = g\beta(T_h - T_c)L^3/(\nu\alpha)$, the strength of buoyancy against friction and heat diffusion, and the **Prandtl number** $Pr = \nu/\alpha = 0.71$ for air.

```
Pr = 0.71                       # Prandtl number of air
Ra_values = [1e3, 1e4]          # the two Rayleigh numbers of the problem

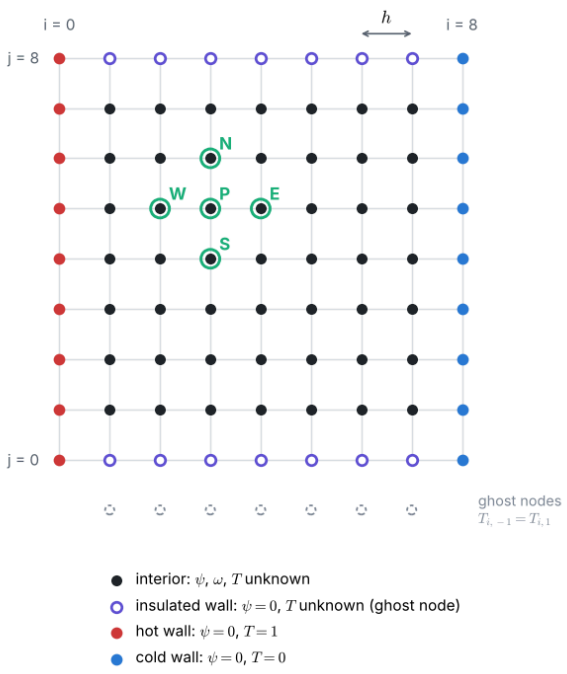
# a real example: the air gap of a double-glazed window
g, beta = 9.81, 1 / 300         # gravity [m/s^2], expansion coefficient of air at ~300 K [1/K]
nu_air, alpha_air = 1.57e-5, 2.21e-5   # air at ~300 K [m^2/s]
dT = 10.0                       # temperature difference between the panes [K]
for L_gap in (0.010, 0.022):
    Ra_gap = g * beta * dT * L_gap**3 / (nu_air * alpha_air)
    print(f"air gap L = {L_gap * 100:.1f} cm, dT = {dT:g} K:  Ra = {Ra_gap:,.0f}")
```

```
air gap L = 1.0 cm, dT = 10 K:  Ra = 942
air gap L = 2.2 cm, dT = 10 K:  Ra = 10,035
```

So $Ra = 10^3$ and 10^4 correspond to double-glazing gaps of about 1 cm and 2 cm with 10 K between the panes.

Step 2 · The grid

The grid of Problem 6: $N = 32$ intervals per side, $h = 1/32$. ψ and ω are unknown at the interior nodes; T is also unknown on the insulated top and bottom walls.



The grid (drawn with $N = 8$): hot and cold walls with known T, insulated walls with unknown T and ghost nodes below them, interior nodes, and one five-point stencil.

```
N = 32                  # intervals per side
h = 1 / N               # grid spacing (in units of L)
x = [i * h for i in range(N + 1)]

print(f"N = {N}, h = {h:.5f}: unknowns per variable: psi and w {(N - 1) ** 2}, T {(N - 1) * (N + 1)}")
```

```
N = 32, h = 0.03125: unknowns per variable: psi and w 961, T 1023
```

Step 3 · Physics: stream function, vorticity and temperature

The flow equations of Problem 6 get two additions:

- **Buoyancy** (Boussinesq approximation): the density changes slightly with temperature, which matters only in the gravity term. Taking the curl of the momentum equations turns the buoyancy force into a **source of vorticity** $Ra\,Pr\,\partial T/\partial x$: wherever warm and cold fluid sit side by side, the fluid starts to turn.
- **Energy:** the temperature is carried by the flow and spreads by conduction, a convection–diffusion equation (Problem 5 in 2D).

$$\nabla^2\psi = -\omega, \qquad u\,\omega_x + v\,\omega_y = Pr\,\nabla^2\omega + Ra\,Pr\,T_x, \qquad u\,T_x + v\,T_y = \nabla^2 T.$$

Wall	Flow	Temperature
left	$\psi = 0$, Thom's formula	$T = 1$ (hot)
right	$\psi = 0$, Thom's formula	$T = 0$ (cold)
top, bottom	$\psi = 0$, Thom's formula	$\partial T/\partial y = 0$ (insulated)

Heat transfer. The local Nusselt number $Nu(y) = -\partial T/\partial x$ on the hot wall is the wall heat flux divided by the flux of pure conduction; its average $\overline{Nu}$ tells how many times more heat crosses the cavity than by conduction alone ($\overline{Nu} = 1$ without flow).

Benchmark. de Vahl Davis (1983) obtained reference values by extrapolating results from several fine grids:

```
# de Vahl Davis (1983), Pr = 0.71: |psi| at the centre, the largest u on the line x = 0.5
# (and its height y), the largest v on the line y = 0.5 (and its position x), and the average Nu
davis = {1e3: {"psi_mid": 1.174, "u_max": (3.649, 0.813), "v_max": (3.697, 0.178), "Nu": 1.118},
         1e4: {"psi_mid": 5.071, "u_max": (16.178, 0.823), "v_max": (19.617, 0.119), "Nu": 2.243}}

for Ra, d in davis.items():
    print(f"Ra = {Ra:.0e}: |psi_mid| = {d['psi_mid']}, u_max = {d['u_max'][0]} at y = {d['u_max']
[1]}, "
          f"v_max = {d['v_max'][0]} at x = {d['v_max'][1]}, Nu = {d['Nu']}")
```

```
Ra = 1e+03: |psi_mid| = 1.174, u_max = 3.649 at y = 0.813, v_max = 3.697 at x = 0.178, Nu = 1.118
Ra = 1e+04: |psi_mid| = 5.071, u_max = 16.178 at y = 0.823, v_max = 19.617 at x = 0.119, Nu =
2.243
```

Step 4 · Numerical scheme

For an interior node P with neighbours E, W, N, S (central differences everywhere):

$$\psi_P = \tfrac{1}{4}\left(\psi_E + \psi_W + \psi_N + \psi_S + h^2\,\omega_P\right),$$

$$\omega_P = \tfrac{1}{4}\left[(1-a_x)\,\omega_E + (1+a_x)\,\omega_W + (1-a_y)\,\omega_N + (1+a_y)\,\omega_S + \tfrac{Ra\,h}{2}\,(T_E - T_W)\right],$$

$$T_P = \tfrac{1}{4}\left[(1-b_x)\,T_E + (1+b_x)\,T_W + (1-b_y)\,T_N + (1+b_y)\,T_S\right],$$

with $a_x = u_P h/(2Pr) = (\psi_N - \psi_S)/(4Pr)$, $a_y = (\psi_W - \psi_E)/(4Pr)$,
$b_x = u_P h/2 = (\psi_N - \psi_S)/4$ and $b_y = (\psi_W - \psi_E)/4$.

Insulated walls (ghost nodes, Problem 1). On the bottom wall $\partial T/\partial y \approx (T_{i,1} - T_{i,-1})/(2h) = 0$ gives $T_{i,-1} = T_{i,1}$. With $u = v = 0$ on the wall the temperature equation there is pure conduction:

$$T_{i,0} = \tfrac{1}{4}(T_{i+1,0} + T_{i-1,0} + 2\,T_{i,1}), \qquad \text{and likewise on the topwall.}$$

Wall vorticity. Thom's formula of Problem 6 for walls at rest, $\omega_{wall} = -2\psi_{adjacent}/h^2$:

```
def wall_vorticity(psi, w, N, h):
    """Thom's formula on the four walls at rest."""
    for k in range(N + 1):
        w[0][k] = -2 * psi[1][k] / h**2          # bottom
        w[N][k] = -2 * psi[N - 1][k] / h**2      # top
        w[k][0] = -2 * psi[k][1] / h**2          # left (hot)
        w[k][N] = -2 * psi[k][N - 1] / h**2      # right (cold)
```

Step 5 · Solution setup

Start from **pure conduction**: $T = 1 - x$, $\psi = \omega = 0$. Then repeat, always using the newest values (Gauss–Seidel):

1. wall vorticity (Thom);
2. one sweep of the temperature equation, including the insulated walls;
3. one sweep of the vorticity equation, with the buoyancy source from the new T;
4. one sweep of the stream-function equation;

until the largest changes of T, ω and ψ are all below 10^{-6}.

```python
def solve_convection(N, Ra, tol=1e-6, max_iter=200_000):
    """Steady natural convection in a square cavity: Gauss-Seidel on T, w and psi."""
    h = 1 / N
    psi = [[0.0] * (N + 1) for _ in range(N + 1)]
    w = [[0.0] * (N + 1) for _ in range(N + 1)]
    T = [[1 - i * h for i in range(N + 1)] for _ in range(N + 1)]   # start: pure conduction
    changes = []                    # largest change of (T, w, psi) in every iteration

    for iteration in range(max_iter):
        # step 1: vorticity on the walls (Thom's formula)
        wall_vorticity(psi, w, N, h)

        # step 2: temperature sweep (rows 0 and N are the insulated walls)
        largest_T = 0.0
        for j in range(N + 1):
            for i in range(1, N):
                if j == 0:                                  # bottom wall, ghost node T[-1] =
T[1]
                    new = (T[0][i + 1] + T[0][i - 1] + 2 * T[1][i]) / 4
                elif j == N:                                # top wall, ghost node T[N+1] = T[N-
1]
                    new = (T[N][i + 1] + T[N][i - 1] + 2 * T[N - 1][i]) / 4
                else:
                    bx = (psi[j + 1][i] - psi[j - 1][i]) / 4          # u h / 2
                    by = (psi[j][i - 1] - psi[j][i + 1]) / 4          # v h / 2
                    new = ((1 - bx) * T[j][i + 1] + (1 + bx) * T[j][i - 1]
                           + (1 - by) * T[j + 1][i] + (1 + by) * T[j - 1][i]) / 4
                largest_T = max(largest_T, abs(new - T[j][i]))
                T[j][i] = new

        # step 3: vorticity sweep, with the buoyancy source Ra h (T_E - T_W) / 2
        largest_w = 0.0
        for j in range(1, N):
            for i in range(1, N):
                ax = (psi[j + 1][i] - psi[j - 1][i]) / (4 * Pr)       # u h / (2 Pr)
                ay = (psi[j][i - 1] - psi[j][i + 1]) / (4 * Pr)       # v h / (2 Pr)
                new = ((1 - ax) * w[j][i + 1] + (1 + ax) * w[j][i - 1]
                       + (1 - ay) * w[j + 1][i] + (1 + ay) * w[j - 1][i]
                       + Ra * h / 2 * (T[j][i + 1] - T[j][i - 1])) / 4
                largest_w = max(largest_w, abs(new - w[j][i]))
                w[j][i] = new

        # step 4: stream function sweep (five-point formula)
        largest_psi = 0.0
        for j in range(1, N):
            for i in range(1, N):
                new = (psi[j][i + 1] + psi[j][i - 1] + psi[j + 1][i] + psi[j - 1][i]
                       + h * h * w[j][i]) / 4
                largest_psi = max(largest_psi, abs(new - psi[j][i]))
                psi[j][i] = new

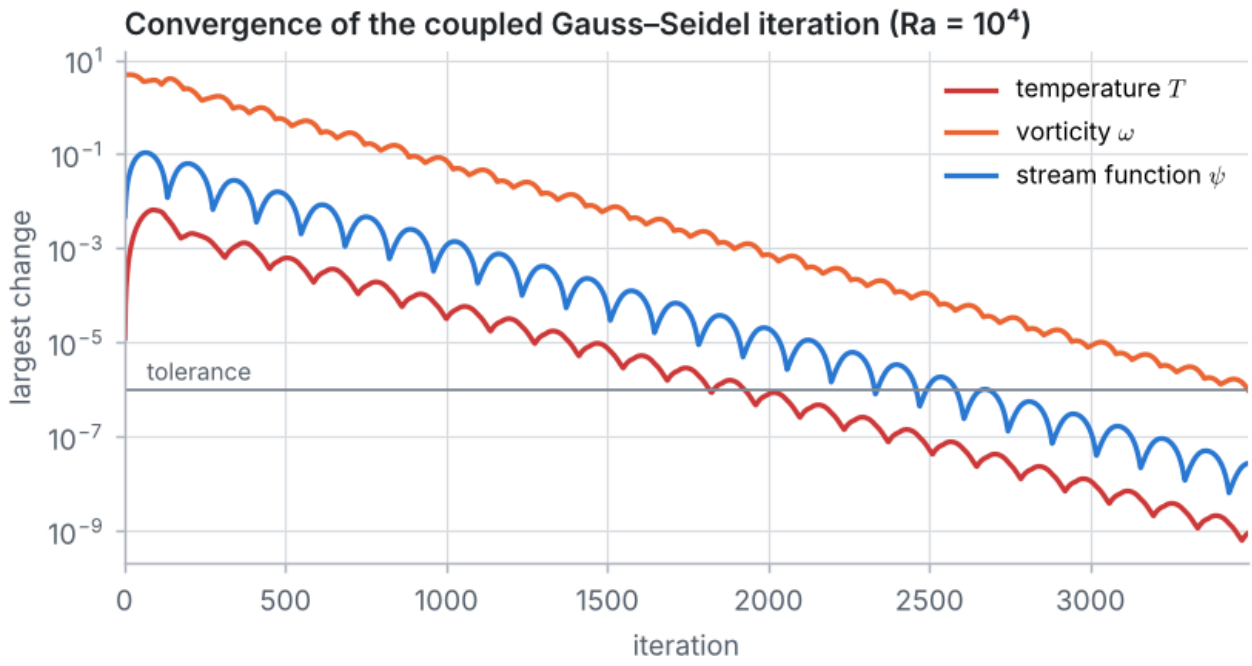
        if not math.isfinite(psi[N // 2][N // 2]):                # the numbers blew up
            raise ArithmeticError(f"the iteration diverged after {iteration + 1} iterations")
        changes.append((largest_T, largest_w, largest_psi))
        if max(largest_T, largest_w, largest_psi) < tol:          # converged
            break

    return psi, w, T, changes
```

Step 6 · Compute

```
solutions = {}
for Ra in Ra_values:
    start = time.time()
    psi, w, T, changes = solve_convection(N, Ra)
    solutions[Ra] = (psi, w, T, changes)
    print(f"Ra = {Ra:.0e}: converged after {len(changes)} iterations ({time.time() - start:.1f}
s)")
```

```
Ra = 1e+03: converged after 1450 iterations (1.5 s)
Ra = 1e+04: converged after 3489 iterations (3.3 s)
```



Largest change per iteration for $Ra = 10^4$. The three changes fall together, in slow waves, below the tolerance.

In iteration 1 the temperature does not change at all: the conduction profile already satisfies the energy equation while the fluid is at rest. Buoyancy creates vorticity, and from iteration 2 on the moving fluid carries the heat.

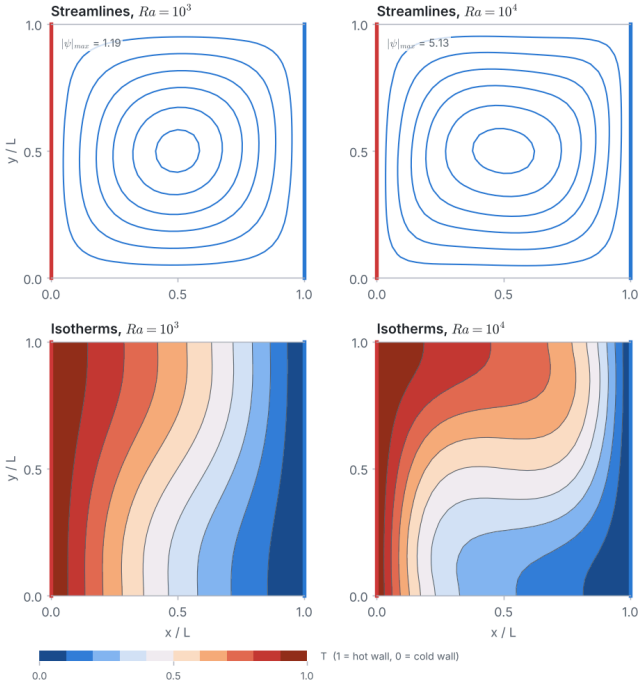
The whiteboard numbers. The whiteboard guide does the first iterations by hand for $Ra = 10^3$ on $N = 4$. Because the iteration always starts from the same conduction state, `max_iter=1` returns the fields after one iteration:

```
def show(field, digits):
    """Print a field row by row, from the top down (+ 0.0 turns -0.0 into 0.0)."""
    for j in range(len(field) - 1, -1, -1):
        print(f"   j = {j}: " + "  ".join(f"{value + 0.0:{digits + 5}.{digits}f}" for value in
field[j]))


for label, iterations in (("after iteration 1", 1), ("after iteration 2", 2), ("converged",
200_000)):
    psi_h, w_h, T_h, changes_h = solve_convection(4, 1e3, max_iter=iterations)
    print(f"N = 4, Ra = 1e3, {label} ({len(changes_h)} iterations)")
    for name, field, digits in (("temperature T", T_h, 4), ("vorticity w", w_h, 4), ("stream
function psi", psi_h, 4)):
        print(f"  {name}:")
        show(field, digits)
```

```
N = 4, Ra = 1e3, after iteration 1 (1 iterations)
  temperature T:
   j = 4:    1.0000     0.7500     0.5000     0.2500     0.0000
   j = 3:    1.0000     0.7500     0.5000     0.2500     0.0000
   j = 2:    1.0000     0.7500     0.5000     0.2500     0.0000
   j = 1:    1.0000     0.7500     0.5000     0.2500     0.0000
   j = 0:    1.0000     0.7500     0.5000     0.2500     0.0000
  vorticity w:
   j = 4:    0.0000     0.0000     0.0000     0.0000     0.0000
   j = 3:    0.0000   -20.5078   -27.0996   -29.1748     0.0000
   j = 2:    0.0000   -19.5312   -25.3906   -27.0996     0.0000
   j = 1:    0.0000   -15.6250   -19.5312   -20.5078     0.0000
   j = 0:    0.0000     0.0000     0.0000     0.0000     0.0000
  stream function psi:
   j = 4:    0.0000     0.0000     0.0000     0.0000     0.0000
   j = 3:    0.0000    -0.4120    -0.6714    -0.7915     0.0000
   j = 2:    0.0000    -0.3662    -0.5798    -0.6714     0.0000
   j = 1:    0.0000    -0.2441    -0.3662    -0.4120     0.0000
   j = 0:    0.0000     0.0000     0.0000     0.0000     0.0000
N = 4, Ra = 1e3, after iteration 2 (2 iterations)
  temperature T:
   j = 4:    1.0000     0.7545     0.5091     0.2629     0.0000
   j = 3:    1.0000     0.7590     0.5160     0.2713     0.0000
   j = 2:    1.0000     0.7415     0.4830     0.2289     0.0000
   j = 1:    1.0000     0.7386     0.4794     0.2247     0.0000
   j = 0:    1.0000     0.7500     0.5000     0.2500     0.0000
  vorticity w:
   j = 4:    0.0000    13.1836    21.4844    25.3296     0.0000
   j = 3:   13.1836   -24.3285   -36.3624   -20.7506    25.3296
   j = 2:   11.7188   -30.8651   -44.6237   -31.6926    21.4844
   j = 1:    7.8125   -22.1271   -29.5025   -21.3168    13.1836
   j = 0:    0.0000     7.8125    11.7188    13.1836     0.0000
  stream function psi:
   j = 4:    0.0000     0.0000     0.0000     0.0000     0.0000
   j = 3:    0.0000    -0.7636    -1.3217    -0.9635     0.0000
   j = 2:    0.0000    -0.8624    -1.4588    -1.2356     0.0000
   j = 1:    0.0000    -0.5288    -0.8411    -0.7112     0.0000
   j = 0:    0.0000     0.0000     0.0000     0.0000     0.0000
N = 4, Ra = 1e3, converged (45 iterations)
  temperature T:
   j = 4:    1.0000     0.7994     0.5842     0.3204     0.0000
   j = 3:    1.0000     0.8067     0.6085     0.3487     0.0000
   j = 2:    1.0000     0.7255     0.5000     0.2745     0.0000
   j = 1:    1.0000     0.6513     0.3915     0.1933     0.0000
   j = 0:    1.0000     0.6796     0.4158     0.2006     0.0000
  vorticity w:
   j = 4:    0.0000    24.0282    36.6653    26.6911     0.0000
   j = 3:   24.0282   -10.9789   -20.5625   -16.3046    26.6911
   j = 2:   37.4899   -22.2116   -35.4786   -22.2116    37.4899
   j = 1:   26.6911   -16.3046   -20.5625   -10.9789    24.0282
   j = 0:    0.0000    26.6911    36.6653    24.0282     0.0000
  stream function psi:
   j = 4:    0.0000     0.0000     0.0000     0.0000     0.0000
   j = 3:    0.0000    -0.7509    -1.1458    -0.8341     0.0000
   j = 2:    0.0000    -1.1716    -1.7130    -1.1716     0.0000
   j = 1:    0.0000    -0.8341    -1.1458    -0.7509     0.0000
   j = 0:    0.0000     0.0000     0.0000     0.0000     0.0000
```

Step 7 · Post-processing and verification



Streamlines (top) and isotherms (bottom). At $Ra = 10^3$ the isotherms are almost the straight lines of pure conduction; at $Ra = 10^4$ the circulation is about four times stronger and the isotherms are strongly tilted: thin thermal layers along the side walls and a stratified core.

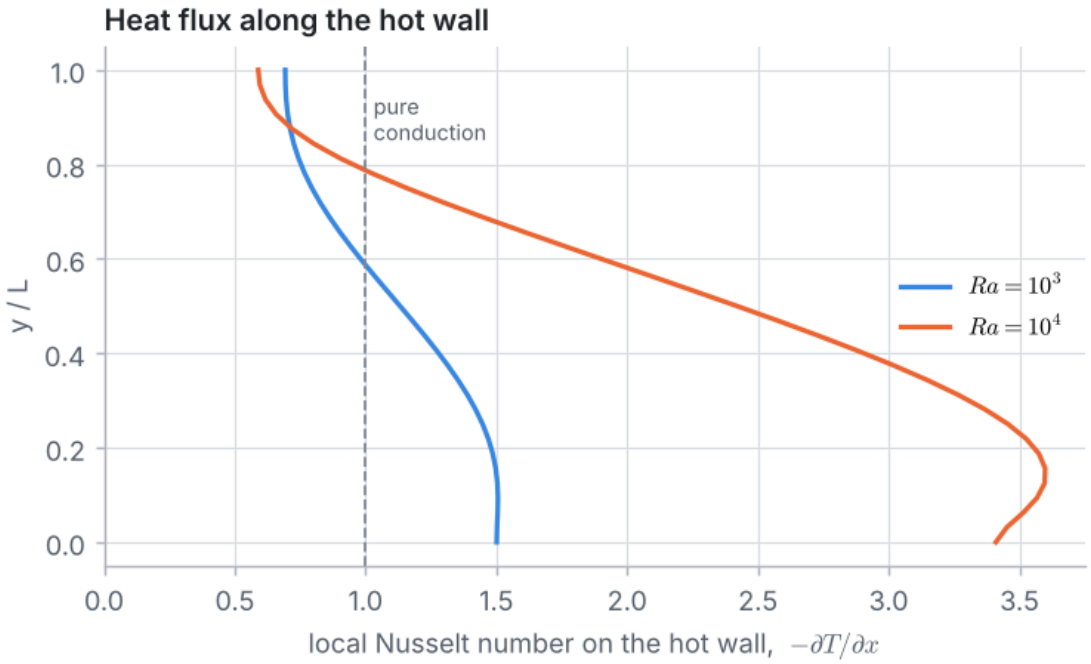
Nusselt number and energy balance. On the walls only nodes inside the cavity are available, so the wall gradient uses a one-sided second-order difference, $\partial T/\partial x|_{x=0} \approx (-3T_0 + 4T_1 - T_2)/(2h)$ (and the mirror image on the cold wall), averaged with the trapezoidal rule:

```
def nusselt(T, N):
    """Local Nu along the hot and cold walls, and their averages (trapezoidal rule)."""
    h = 1 / N
    hot = [-(-3 * T[j][0] + 4 * T[j][1] - T[j][2]) / (2 * h) for j in range(N + 1)]
    cold = [-(3 * T[j][N] - 4 * T[j][N - 1] + T[j][N - 2]) / (2 * h) for j in range(N + 1)]
    average = lambda f: h * (sum(f) - (f[0] + f[-1]) / 2)
    return hot, cold, average(hot), average(cold)


for Ra in Ra_values:
    hot, cold, Nu_hot, Nu_cold = nusselt(solutions[Ra][2], N)
    j_max = max(range(N + 1), key=lambda j: hot[j])
    print(f"Ra = {Ra:.0e}: Nu hot wall {Nu_hot:.4f}, cold wall {Nu_cold:.4f}, difference {Nu_hot -
Nu_cold:+.4f};"
          f"  largest local Nu {hot[j_max]:.3f} at y = {j_max * h:.3f}")
```

```
Ra = 1e+03: Nu hot wall 1.1181, cold wall 1.1181, difference +0.0000;  largest local Nu 1.506 at y
= 0.094
Ra = 1e+04: Nu hot wall 2.2681, cold wall 2.2681, difference +0.0000;  largest local Nu 3.598 at y
= 0.156
```

The hot-wall and cold-wall averages agree to all printed decimals, as they must: energy is conserved (the top and bottom are insulated), and the problem is symmetric (turning the cavity upside down and swapping hot and cold gives the same problem).



Local Nusselt number along the hot wall. The heat flux is largest near the bottom, where cold fluid arriving along the bottom meets the hot wall, and smallest at the top.

Grid study. Both Rayleigh numbers on $N = 8$ to 64 (the finest grid takes up to a minute per case in plain Python):

```
def key_values(psi, T, N):
    """|psi| at the centre, largest u on x = 0.5 and v on y = 0.5 (with positions), and Nu."""
    h = 1 / N
    m = N // 2
    u_line = [((psi[j + 1][m] - psi[j - 1][m]) / (2 * h), j * h) for j in range(1, N)]
    v_line = [(-(psi[m][i + 1] - psi[m][i - 1]) / (2 * h), i * h) for i in range(1, N)]
    return {"psi_mid": abs(psi[m][m]), "u_max": max(u_line), "v_max": max(v_line),
            "Nu": nusselt(T, N)[2]}
```

```
grids = [8, 16, 32, 64]
study = {}
for Ra in Ra_values:
    print(f"Ra = {Ra:.0e}")
    print("    N   iterations  time [s]   |psi_mid|    u_max (y)          v_max (x)          Nu")
    study[Ra] = {}
    for n_int in grids:
        start = time.time()
        psi_n, w_n, T_n, changes_n = solve_convection(n_int, Ra)
        k = key_values(psi_n, T_n, n_int)
        study[Ra][n_int] = k
        print(f"{n_int:5d}   {len(changes_n):8d}  {time.time() - start:8.1f}
{k['psi_mid']:8.4f}"
              f"   {k['u_max'][0]:7.3f} ({k['u_max'][1]:.3f})   {k['v_max'][0]:7.3f} ({k['v_max']
[1]:.3f})"
              f"   {k['Nu']:.4f}")
    d = davis[Ra]
    print(f"  de Vahl Davis          {d['psi_mid']:8.4f}   {d['u_max'][0]:7.3f} ({d['u_max']
[1]:.3f})"
          f"   {d['v_max'][0]:7.3f} ({d['v_max'][1]:.3f})   {d['Nu']:.4f}")
```

```
Ra = 1e+03
    N   iterations  time [s]   |psi_mid|    u_max (y)          v_max (x)          Nu
    8        139       0.0     1.3321     3.418 (0.750)     3.411 (0.125)   1.1365
   16        445       0.1     1.2160     3.649 (0.812)     3.686 (0.188)   1.1209
   32       1450       1.5     1.1851     3.649 (0.812)     3.690 (0.188)   1.1181
   64       4510      20.1     1.1773     3.649 (0.812)     3.694 (0.172)   1.1178
  de Vahl Davis            1.1740     3.649 (0.813)     3.697 (0.178)   1.1180
Ra = 1e+04
    N   iterations  time [s]   |psi_mid|    u_max (y)          v_max (x)          Nu
    8        910       0.1     5.8428    15.080 (0.875)    18.935 (0.125)   2.8088
   16       1068       0.3     5.2880    16.209 (0.812)    19.394 (0.125)   2.3794
   32       3489       3.8     5.1288    16.165 (0.812)    19.544 (0.125)   2.2681
   64      12493      60.7     5.0876    16.180 (0.828)    19.583 (0.125)   2.2487
  de Vahl Davis            5.0710    16.178 (0.823)    19.617 (0.119)   2.2430
```

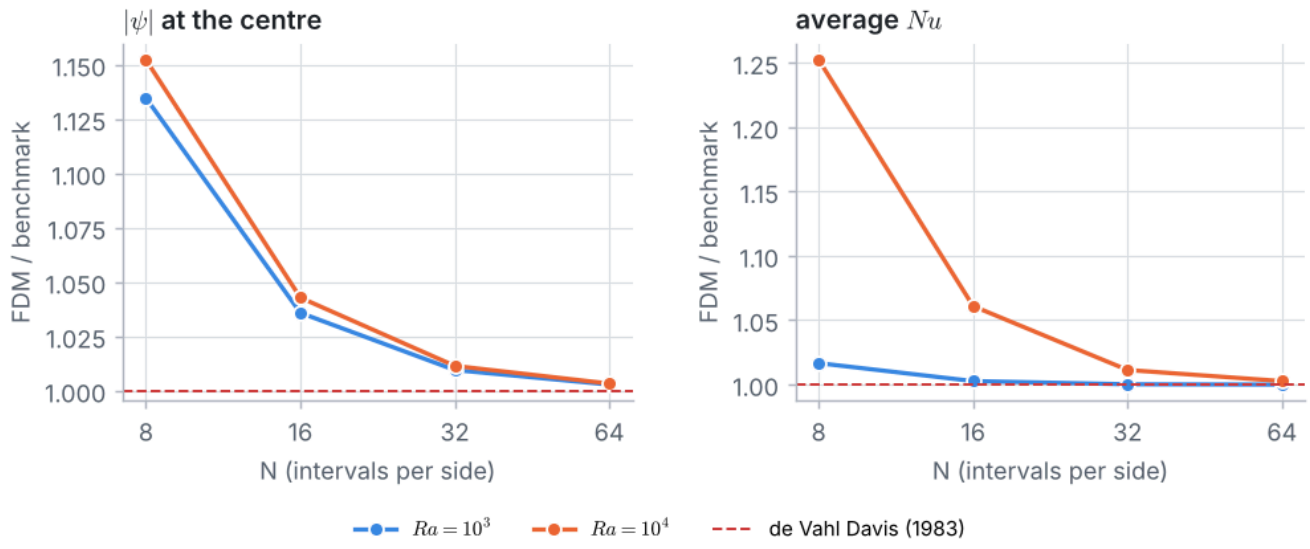
Richardson extrapolation. For a second-order scheme the error behaves like Ch^2, so two grids give an estimate of the exact value,

$$f_{exact} \approx f_{h/2} + \frac{f_{h/2} - f_h}{3},$$

and three grids give the observed order $p = \log_2\left[(f_{16} - f_{32})/(f_{32} - f_{64})\right]$:

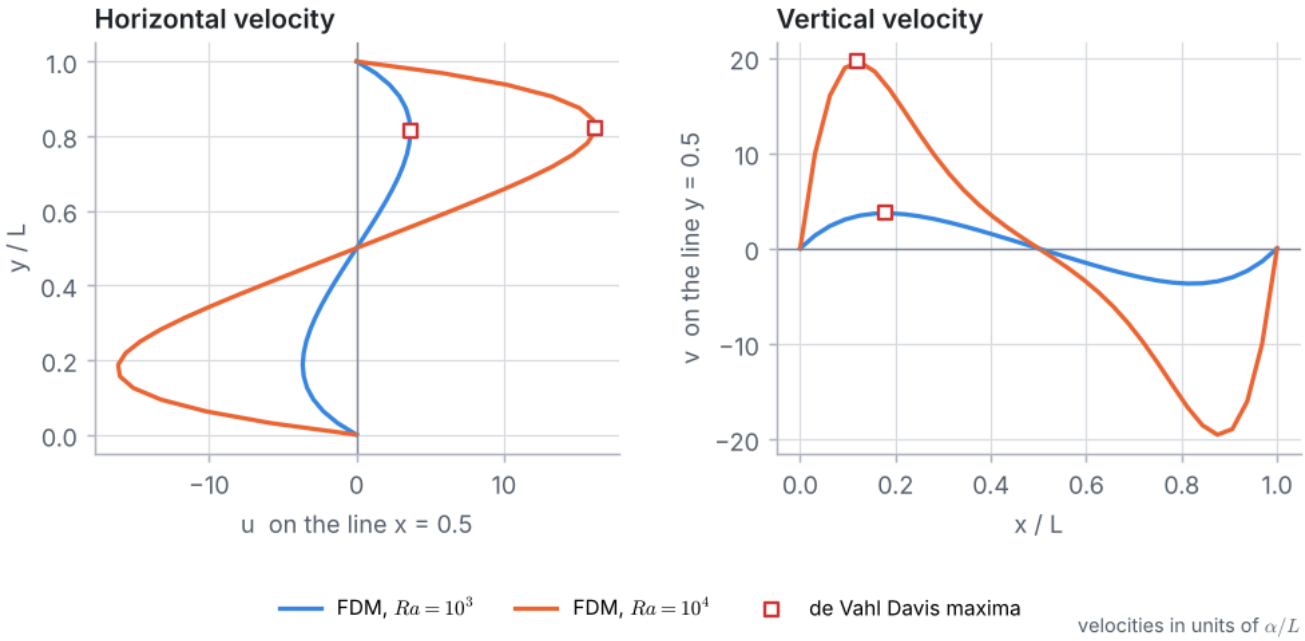
```python
print("             N = 64     extrapolated   de Vahl Davis   difference   observed order")
for Ra in Ra_values:
    s = study[Ra]
    for name in ("psi_mid", "Nu"):
        f16, f32, f64 = s[16][name], s[32][name], s[64][name]
        extrapolated = f64 + (f64 - f32) / 3
        order = math.log2((f16 - f32) / (f32 - f64))
        reference = davis[Ra][name]
        print(f"Ra = {Ra:.0e} {name:8s} {f64:8.4f}    {extrapolated:8.4f}       {reference:8.4f}"
              f"     {100 * (extrapolated - reference) / reference:+.2f} %       {order:.2f}")
```

```
             N = 64     extrapolated   de Vahl Davis   difference   observed order
Ra = 1e+03 psi_mid    1.1773      1.1747         1.1740     +0.06 %       1.97
Ra = 1e+03 Nu         1.1178      1.1177         1.1180     -0.02 %       3.30
Ra = 1e+04 psi_mid    5.0876      5.0738         5.0710     +0.06 %       1.95
Ra = 1e+04 Nu         2.2487      2.2423         2.2430     -0.03 %       2.52
```



Results on each grid divided by the benchmark value: both quantities approach the de Vahl Davis results as the grid is refined.

- $|\psi_{mid}|$ converges with an observed order close to 2. For $\overline{Nu}$ the observed order is higher (3.3 and 2.5): its changes are already very small, so its error is not yet the clean Ch^2 of the theory.
- The extrapolated values agree with the de Vahl Davis benchmark to within 0.1 %, from grids of at most 65 × 65 nodes.



Velocity along the two centre lines (in units of α/L). The squares are the maxima given by de Vahl Davis; at $Ra = 10^4$ the vertical velocity peaks in thin, fast layers next to the walls.

Result. At $Ra = 10^3$ the circulation is weak ($|\psi_{mid}| = 1.185$ on 33 × 33 nodes) and heat crosses the cavity mostly by conduction: $\overline{Nu} = 1.118$. At $Ra = 10^4$ the flow is about four times stronger ($|\psi_{mid}| = 5.129$) and more than doubles the heat transfer: $\overline{Nu} = 2.268$ on 33 × 33 nodes, 2.249 on 65 × 65 nodes, and 2.242 after Richardson extrapolation, against 2.243 from de Vahl Davis.

Key ideas

- Buoyancy (Boussinesq) adds a source $Ra\,Pr\,\partial T/\partial x$ to the vorticity equation; the energy equation is a convection–diffusion equation. Flow and heat are coupled both ways.
- Insulated walls are applied with ghost nodes; walls at rest get their vorticity from Thom's formula.
- Three coupled equations can be solved with one Gauss–Seidel iteration, starting from pure conduction.
- The Nusselt number measures how much the flow increases the heat transfer; the energy balance checks it.
- Richardson extrapolation turns two grids into an estimate of the grid-independent answer.